

ABC457 G - Catch All Apples 题解

题意概括

数轴上有 N 个苹果，第 i 个苹果会在时刻 T_i 掉到坐标 X_i 。

每个机器人从时刻 0 开始工作，可以放在任意初始位置，并且移动速度不超过 1。当机器人在时刻 T_i 恰好到达坐标 X_i 时，就能接住这个苹果。

要求用尽量少的机器人接住全部苹果。

解题思路

1. 先把“能否连续接住两个苹果”写成不等式

如果同一个机器人先接住苹果 i ，再接住苹果 j ，那么一定有 $T_i \leq T_j$ ，并且它需要在时间差 $T_j - T_i$ 内，从 X_i 走到 X_j ，所以必须满足：

- $X_j - X_i \leq T_j - T_i$
- $X_i - X_j \leq T_j - T_i$

整理后得到：

- $T_i + X_i \leq T_j + X_j$
- $T_i - X_i \leq T_j - X_j$

于是定义：

- $u_i = T_i + X_i$
- $v_i = T_i - X_i$

那么“机器人可以先接住 i 再接住 j ”就等价于：

- $u_i \leq u_j$
- $v_i \leq v_j$

也就是说，一个机器人接到的一串苹果，在 (u, v) 平面上必须同时满足两个坐标都不下降。

2. 按 (u, v) 排序后，问题变成最少划分多少个非递减子序列

把所有苹果按 (u, v) 从小到大排序。

排序后，如果一组苹果能交给同一个机器人，那么它们的 u 一定已经按顺序不下降，因此只需要再保证对应的 v 也不下降即可。

所以原问题等价于：

- 把排序后的 $v_1, v_2, \dots, v_N$
- 划分成尽量少个非递减子序列

每个子序列对应一个机器人。

3. 用贪心维护每个机器人的“最后一个 v ”

依次处理排好序的每个 v 。

假设当前已经开了一些机器人，我们只关心每个机器人最后接到的苹果的 v 值。设这些值组成一个集合。

处理当前 v 时：

1. 如果存在某个机器人最后的 v 不大于当前 v ，那么这个机器人就能继续接当前苹果。
2. 为了给后面的苹果留下尽量多的选择，应该把当前苹果接到“最后的 v 最大但仍然不超过当前 v ”的那个机器人后面。
3. 如果所有机器人的最后 v 都大于当前 v ，说明当前苹果不能接到任何已有机器人后面，只能新开一个机器人。

这就是经典的“最少非递减子序列覆盖”贪心。

这里苹果本身可以先读进普通数组，再排序后依次处理；但机器人“当前最后一个 v ”这批值会动态变化，并且需要反复完成下面两件事：

- 找到最大的、满足 $\leq$ 当前 v 的末尾值；
- 删掉这个旧末尾，再插入新的末尾值。

如果只用普通数组保存这些末尾值，不管写成无序数组还是有序数组，都会在“查找前驱”或“删除插入并保持有序”上退化到 $O(N)$ 。由于 $N \leq 3 \times 10^5$ ，这里需要保留 multiset，用它在 $O(\log N)$ 内完成查找、删除和插入。

正确性说明

性质 1

对任意两个苹果 i, j ，若 $T_i \leq T_j$ ，则同一个机器人可以先接住 i 再接住 j ，当且仅当 $u_i \leq u_j$ 且 $v_i \leq v_j$ 。

原因在解题思路第一部分已经推导过：它与速度限制 $|X_j - X_i| \leq T_j - T_i$ 完全等价。

性质 2

把所有苹果按 (u, v) 升序排序后，一个机器人能接住的一组苹果，恰好对应排序后 v 序列中的一个非递减子序列。

这是因为：

- 排序后，这组苹果的 u 已经天然不下降；
- 根据性质 1，只要对应的 v 也不下降，这个机器人就能依次接住它们；
- 反过来，任何同一机器人接住的苹果序列，都必须同时满足 u 、 v 不下降，因此它一定对应一个非递减子序列。

所以题目已经转化成“最少用多少个非递减子序列覆盖整个序列”。

性质 3

处理当前值 v 时，若存在多个机器人都满足“最后一个值 $\leq v$ ”，选择其中最后一个值最大的那个，一定不劣。

设有两个可选机器人，它们当前最后一个值分别是 a 和 b ，且 $a \leq b \leq v$ 。

如果把当前 v 接在最后值为 a 的机器人后面，那么两个机器人的最后值会变成 v 和 b 。

如果把当前 v 接在最后值为 b 的机器人后面，那么两个机器人的最后值会变成 a 和 v 。

显然保留较小的 a 比保留较大的 b 更有利于将来接更小的值，因此第二种选择不会更差。也就是说，贪心地选“最大的可行末尾”是最优的。

性质 4

如果当前 v 小于所有机器人最后一个值，那么必须新开一个机器人。

因为此时当前 v 不能接在任何已有非递减子序列后面，所以无论怎样安排，都至少需要新增一个子序列，也就是新增一个机器人。

结论

根据性质 2，原问题等价于最少非递减子序列覆盖问题；根据性质 3 和性质 4，上述贪心在每一步都作出了最优且必要的选择。

因此，最终开出的机器人数量就是最少需要的机器人数量，算法正确。

复杂度

- 时间复杂度： $O(N \log N)$
- 空间复杂度： $O(N)$

其中排序需要 $O(N \log N)$ ，multiset 的每次插入、删除、查找都是 $O(\log N)$ 。

参考实现 (C++, 遵守代码规范)

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 300000 + 5;

struct Apple{
    int u, v;
} apple[MAXN];

bool cmp(Apple a, Apple b){
    if(a.u != b.u){
        return a.u < b.u;
    }
    return a.v < b.v;
}

int main(){
    // 读取苹果数量
    int n;
    cin >> n;

    // 读入每个苹果, 并存下变换后的 u_i 和 v_i
    for(int i=1; i≤n; i++){
        int t, x;
        cin >> t >> x;

        apple[i].u = t + x;
        apple[i].v = t - x;
    }

    // 按 (u, v) 从小到大排序
    sort(apple + 1, apple + n + 1, cmp);

    // robot_last 中保存每个机器人当前最后一个 v
    multiset<int> robot_last;

    for(int i=1; i≤n; i++){
        int v = apple[i].v;
```

```
// 找到最后一个  $\leq v$  的机器人
auto it = robot_last.upper_bound(v);

if(it == robot_last.begin()){
    // 没有机器人能接这个苹果，只能新增一个机器人
    robot_last.insert(v);
} else {
    // 选择最后一个值最大的可行机器人，把它的末尾更新成当前 v
    --it;
    robot_last.erase(it);
    robot_last.insert(v);
}

// 机器人数量就是集合中的末尾数量
cout << robot_last.size();

return 0;
}
```